

# CNN, RNN & Regularization

*Two Network Types and the L1 / L2 Technique*

AWS Machine Learning Associate • Part 16

Presented by Prajan Shrestha

# What We'll Cover



The three ideas at a glance



Convolutional Neural Networks



Recurrent Neural Networks



CNN vs. RNN



Regularization: why & what



L1 vs. L2, and in practice

# The Three Ideas at a Glance



### CNN

Built for grid-like data, mostly images



### RNN

Built for sequences where order matters: text, speech, time series



### Regularization

A technique that works with almost any model to reduce overfitting

# CNNs: How They Work

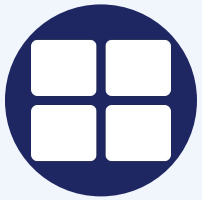
1



## Input Image

A grid of pixel values

2



## Convolution

Filters slide across the image: edges first, then shapes, then objects

3



## Pooling

Shrinks the data, keeping the strongest signals

4



## Fully Connected

Uses the features to decide, like cat or dog

*Early layers find simple patterns. Deeper layers combine them.*

# CNNs: Why They Work Well



### Learns Features Automatically

No need to design edge or shape detectors by hand



### Shares Parameters

The same filter is reused across the whole image, so far fewer weights



### Image Tasks

Classification, object detection, segmentation, medical imaging, face recognition



### In SageMaker

The built-in image classification, object detection & segmentation algorithms are CNN-based

# RNNs: How They Work

1



## Read an Input

One word or one time point at a time

2



## Update Memory

The hidden state summarizes what it has seen so far

3



## Pass It Forward

That memory feeds into the next step

4



## Produce Output

A prediction at each step, or one at the end

*What came before matters: the meaning of a word depends on the words earlier.*

# RNNs: Strengths & Limits



### Built for Sequences

Text, speech and time series data



### The Memory Problem

Memory fades over long sequences (the vanishing gradient problem)



### LSTM & GRU

Gates decide what to keep and what to forget, so they remember longer



### In SageMaker

DeepAR (forecasting) and Seq2Seq (translation) use recurrent models

*Transformers now lead on many language tasks, but RNNs remain common for time series.*

# CNN vs. RNN

| | CNN | RNN |
|---|---|---|
| **Data Type** | Grid-like data, mainly images | Sequences: text, speech, time series |
| **Key Idea** | Filters slide across the input | Memory carried from step to step |
| **Typical Tasks** | Image classification, object detection | Translation, speech recognition, forecasting |
| **In SageMaker** | Image Classification, Object Detection | DeepAR, Seq2Seq |

*A picture? Think CNN. Does the order of the data matter? Think RNN.*

# Regularization: Why & What



**Penalize complexity**

- **Overfitting**
  Great on training data, poor on new data
- **Add a penalty**
  Large weights make the loss bigger
- **Simpler models generalize better**
  Smaller weights work better on unseen data

$$\text{Loss} = \text{Error} + \lambda \times \text{Penalty}$$

λ (lambda) sets how strong the penalty is

# L1 vs. L2 Regularization

## L1 (Lasso)

$$\text{Penalty} = \lambda \times \Sigma\ |w|$$

Pushes some weights to exactly zero, removing those features. Gives sparse models with built-in feature selection.

## L2 (Ridge / Weight Decay)

$$\text{Penalty} = \lambda \times \Sigma\ w^2$$

Shrinks all weights smoothly toward zero, but rarely to exactly zero. Every feature keeps some influence.

*Set λ too high and the model underfits.*

# Regularization in Practice



### Linear Learner

Has hyperparameters for both L1 and L2



### XGBoost

alpha is L1 and lambda is L2



### Dropout

Randomly switches off neurons during training so no single one is relied on



### Early Stopping

Stops training when validation error stops improving

*Tune these values with SageMaker automatic model tuning.*

# Recap

- CNNs use filters to understand images; RNNs carry memory to understand sequences.
- Regularization penalizes large weights to prevent overfitting.
- L1 pushes some weights to zero; L2 shrinks them all smoothly.

**Thanks for watching!**